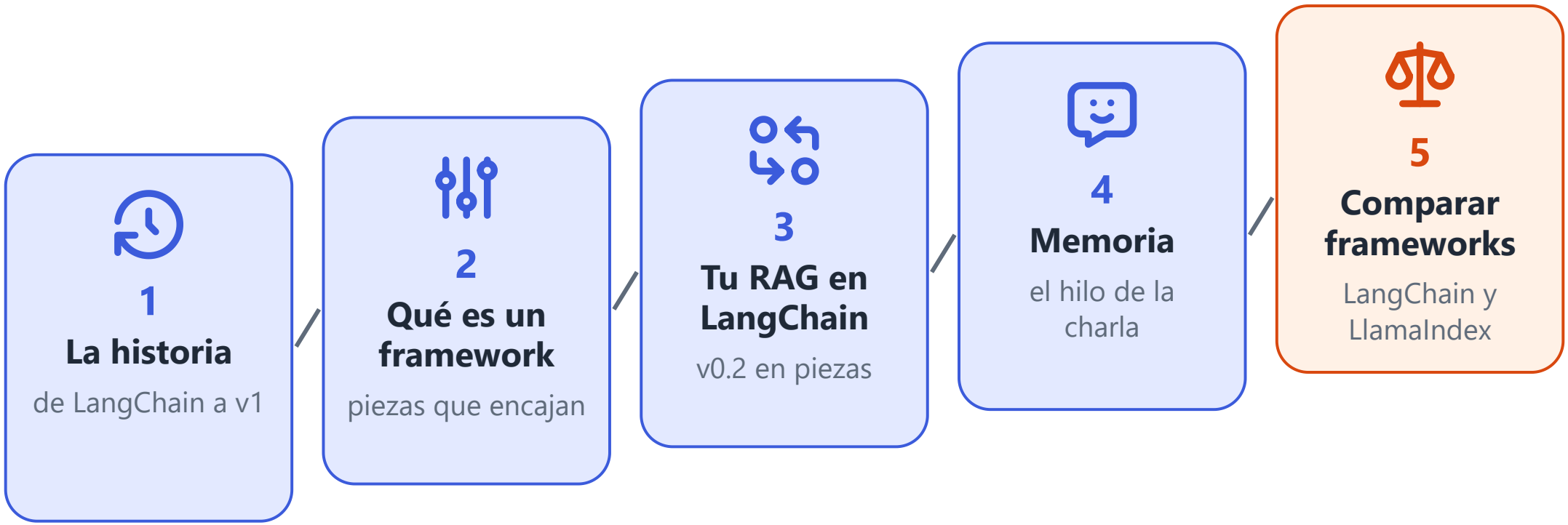
# LangChain y memoria

## M5: v0.3, LangChain y memoria



Hoy: el recorrido

# De código a mano a piezas armadas




Venís de v0.2, armado a mano. Hoy lo pasás a piezas que encajan y le das memoria.

# Primero solo, después lo discutimos

- Cuestionario individual (iRAT), 5 min
- Discusión de las respuestas, 5 min

Sin asistentes de IA. 5 preguntas sobre frameworks, memoria y las lecturas. En la discusión, las mismas.

# Los frameworks crecieron junto con ChatGPT




**Cambian rápido: el proyecto fija las versiones exactas.**

# Una librería da ladrillos, un framework, un kit

**Librería**
**funciones sueltas que llamás cuando querés**

→

**Framework**
**piezas que encajan y una forma de armarlas**

Con el kit vas más rápido, pero no ves todo lo que hace por dentro.

# Un framework te da piezas y te esconde pasos

### Te da

piezas que encajan · memoria y estado · cambiar de proveedor

### Te esconde

prompts internos · llamadas de más · el costo de cada paso

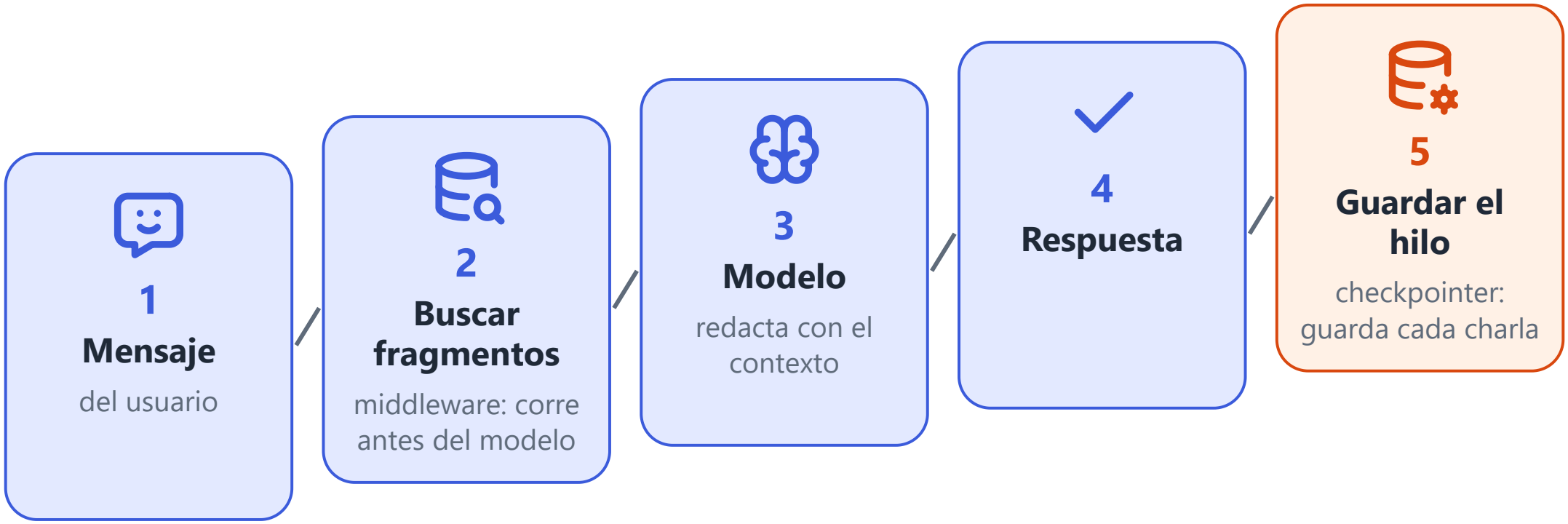
**Tu v0.3 se dibuja «inicio → model → fin»: la búsqueda no se ve.**

# LangChain no es el único kit

- **LangChain**
- **LlamaIndex (demo de hoy)**
- **Haystack**
- **DSPy**
- **Semantic Kernel (Microsoft)**
- **Hugging Face**

Las piezas se parecen: modelo, buscador, prompt, memoria. Cambian los nombres y lo que esconden.

# Tu RAG, armado con piezas de LangChain




Middleware: corre antes del modelo. Checkpointer: guarda cada conversación.

Igual que `v0.2` : busca antes de redactar.

# Una función que corre antes del modelo

```
@dynamic_prompt                                          — Antes de cada llamada
def con_contexto(pedido) -> str:                         — Arma el pedido
    hits = buscar(coleccion, …messages[-1].text, K)      — Busca con el último mensaje
    return f"{SISTEMA_RAG}…{contexto}"                   — Suma el contexto
create_agent(…, middleware=[con_contexto],               — El RAG, como middleware
             checkpointer=InMemorySaver())               — Guarda cada hilo
preguntar(pregunta, hilo)  # thread_id = hilo            — Un hilo por conversación
```

Busca, arma el contexto y guarda cada hilo.

# LCEL: el mismo pedido, como una cadena con |

```
humano = "Contexto:\n{contexto}\n\nPregunta: {pregunta}"
prompt = ChatPromptTemplate.from_messages([("system", SISTEMA), ("human", humano)])
cadena = prompt | cargar_modelo() | StrOutputParser()  # cada | pasa su salida
respuesta = cadena.invoke({"contexto": contexto, "pregunta": pregunta})
```

- Pasos fijos encadenados: LCEL.
- Con gpt-5.4-nano salió `ChatPromptTemplate | ChatOpenAI | StrOutputParser` y citó [langgraph_interrupts]. Con Gemini, el del medio es `ChatGoogleGenerativeAI`

▶ `uv run python -m ejemplos.clase5 lcel`

# LangChain no empeoró la suite

**v0.2, Chroma**
**hit 10/11**

=

**v0.3, LangChain**
**hit 10/11, mismos fragmentos**

Dos corridas (15/10 y 17/10): **sin regresión**. ▶

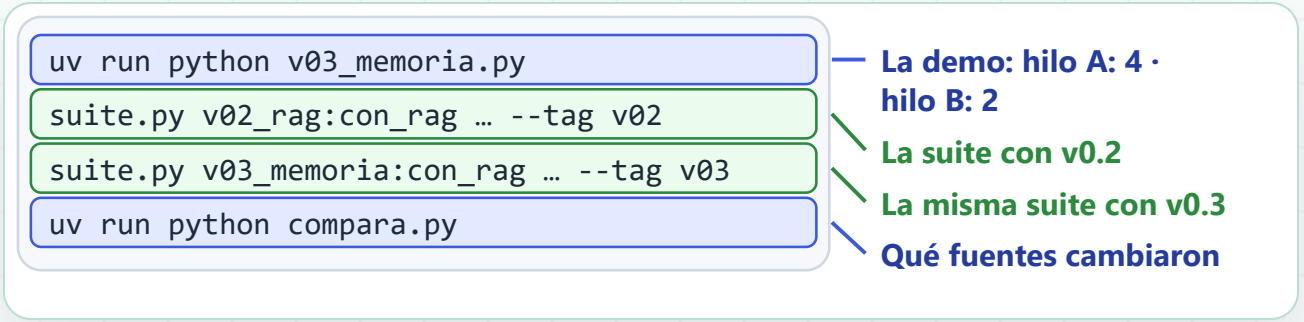
```
uv run python suite.py v03_memoria:con_rag datos/m4_suite.jsonl --tag
v03
```

# Taller: tu RAG en LangChain, medido (50 min)

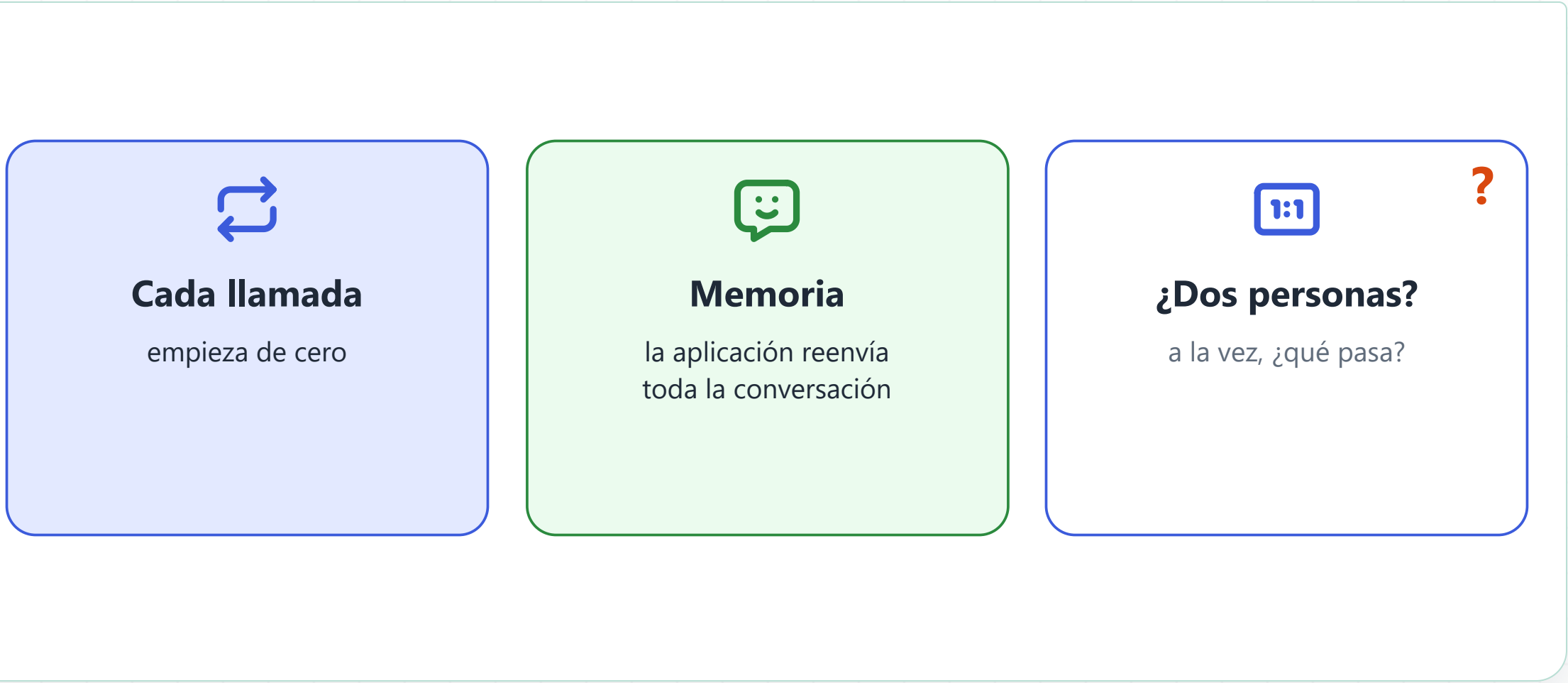
1. **Demo:** `v03_memoria.py`
2. **Suite** con `v0.2` y `v0.3`
3. **Cada diferencia**, con `compara.py`

Práctica M5, arranque y pasos 1 a 3.

**Primer control · 10–15 min:** la demo termina en «hilo A: 4, hilo B: 2».



# La API no recuerda nada entre pedidos




¿Qué pasa si dos personas la usan a la vez?

# Cada conversación guarda solo lo suyo

**Hilo A**
**2 preguntas**
**4 mensajes guardados**

≠

**Hilo B**
**1 pregunta**
**2 mensajes guardados**

- **Hilo** = una conversación, con su propia historia. Cada vuelta guarda 2 mensajes.
- ¿Eso es memoria?

# Dos corridas: el seguimiento acertó una vez

```
corrida 1  Hilo A (seguimiento): NO ENCONTRADO
           Hilo B:               Si el proceso se reinicia, todos los puntos de
                                 control … se pierden [langgraph__persistence].
corrida 2  Hilo A (seguimiento): Si se utiliza MemorySaver o InMemorySaver,
                                 todos los puntos de control se pierden…
           Hilo B:               Cuando el proceso se reinicia, … se pierden…
las dos    Mensajes guardados → hilo A: 4 · hilo B: 2
```

- Gemini 3.5 Flash-Lite.
- La prueba de memoria es la cuenta de mensajes.

▶ `uv run python v03_memoria.py`

# Taller: memoria con tus preguntas (35 min)

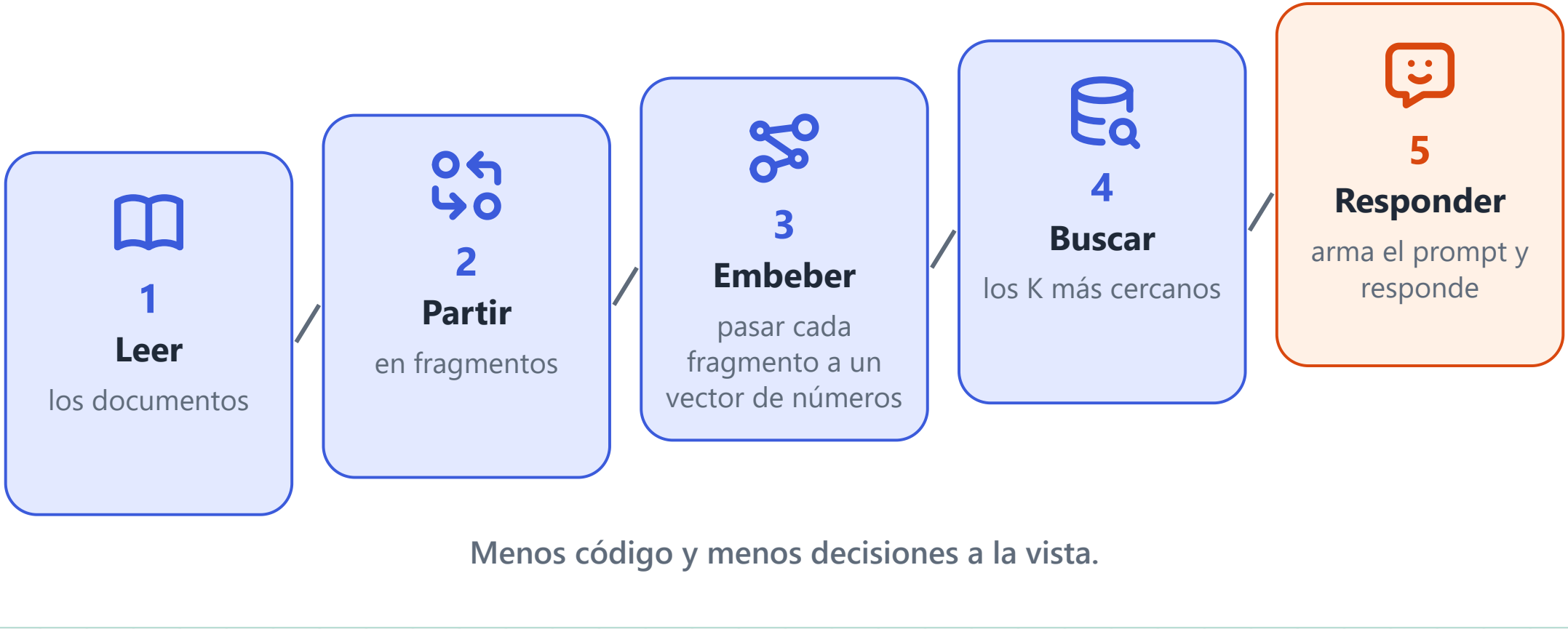
1. **Tu conversación**, con un seguimiento.
2. **Aislamiento:** B no guarda lo de A.
3. **Para casa:** ¿últimos mensajes o resumen?

Práctica M5, pasos 4 a 5.

**Trampa:** la búsqueda ve solo el último mensaje.

```
preguntar("¿Qué guarda…?", "A")      — Tu pregunta, en el hilo A
preguntar("¿Y qué pasa…?", "A")      — Un seguimiento que la necesite
preguntar("¿Y qué pasa…?", "B")      — El mismo seguimiento, en B
mensajes_guardados("B")              — La prueba: B no guarda lo de A
```

# LlamaIndex hace solo lo que en v0.2 escribiste




Menos código y menos decisiones a la vista.

La demo muestra qué decidió por vos.

# El mismo RAG en LlamaIndex son tres líneas

```
docs = SimpleDirectoryReader("corpus").load_data()          # leer
indice = VectorStoreIndex.from_documents(docs)              # partir y embeber, solo
r = indice.as_query_engine(similarity_top_k=4).query(p)     # buscar y responder
```

- Tu `v02_rag.py` hace lo mismo a la vista.
- Solas, piden OpenAI y leen todo `corpus/`, licencia incluida: `ejemplos/clase5.py` antes fija `Settings.llm`, `Settings.embed_model` y los archivos.

# LlamaIndex respondió largo y sin citar

```
Pregunta: ¿Qué hace interrupt en LangGraph?

── LangChain · v02_rag.con_rag (2.3 s)
`interrupt` pausa la ejecución de un grafo en un nodo … y espera una entrada
externa antes de continuar [langgraph__interrupts]. Al llamarlo, … guarda el estado

── LlamaIndex (35.3 s)
Basándose en la información provista, la función `interrupt()` … permite pausar
… 1. **Pausa la ejecución** … 2. **Guarda el estado** … 3. … 4. …   (sin citas)
fuentes: […] · fragmentos: 50
```

- Gemini 3.5 Flash-Lite.
- Los 35 s incluyen rehacer el índice. v0.2 lo reusa.

▶ `uv run --with llama-index-core python -m ejemplos.clase5 comparar`

# Lo que el framework decide por vos

| | v0.2 | LlamaIndex, de fábrica |
|---|---|---|
| Fragmento | 1000 caracteres | 1024 tokens |
| *k* | 4 | 2 |
| Prompt | Tu `SISTEMA` : citar | El suyo, oculto |
| Índice | En `.chroma` | Se rehace cada vez |

Todo se cambia, pero primero hay que saber que existe.

# Corré la comparación con tu pregunta

```
uv run --with llama-index-core python -m ejemplos.clase5 comparar "tu pregunta"
```

- Anotá de cada uno: largo, si cita, tiempo y fuentes.
- ¿Cuál dejarías en tu proyecto, y por qué?

# Repaso: hoy te llevás tres ideas

### Framework

piezas armadas, pasos escondidos

### Memoria

reenviar la conversación

### Medir

igual, con cualquier librería

**Un hilo por persona: cada conversación guarda solo lo suyo.**

# ¿Qué ya no podrías explicar?

Nombrá una parte de tu sistema que ahora hace LangChain y que no sabrías explicar sin leer su código.

Anotá tu **confianza (1 a 4)**.